

AGENTIC ENGINEERING · A ONE-DAY PROGRAM

Welcome

For software engineers returning from carers' or parental leave



ACKNOWLEDGEMENT OF COUNTRY

I would like to acknowledge the Gadigal of the Eora Nation, the traditional custodians of this land and pay my respects to the Elders both past and present.

INTRODUCTIONS



Adair Robins

Tech Council Australia



Jade Loyzaga

Canva



Paul Keen

Culture Amp



Joris Dries

Resonate CX

What is the **one habit** you would love to stick to?



RUN OF DAY

Segment lengths are what matter. Times assume a 09:30 start.

TIME	SEGMENT
09:30	Welcome, introduction, setup check — the day's shape
09:45	How the engineer's life has changed
10:00	Section 1 • Vibe Coding
11:30	Break
11:45	Section 2 • Vibe Engineering
13:15	Lunch
14:00	Section 3 • Agentic Engineering
15:45	Showcase and close
16:00	End



THE PROMISE

What today is really about.

You build **one app, three times over**, each time giving up a different crutch.

The project builds either way, and you learn something new no matter what level you walk in at.

A RUSTY ENGINEER

learns the loop and the tools — and stops fearing them.

A CONFIDENT ONE

learns to give up the keyboard and build the harness instead.

THE DAY IS A LADDER

Not three workshops. Each rung removes a different crutch.

01

Vibe Coding

Use the tool. Make it work, don't care how.

02

Vibe Engineering

Make it repeatable. Stop repeating yourself.

03

Agentic Engineering

Stop trusting the agent. Engineer around it.



ONE PROJECT, ALL DAY: TCA HABITS

A habit tracker you build, then make repeatable, then harden.

END OF SECTION 1

A working tracker. Add a habit, tick it, watch a streak grow. Ugly is fine.



END OF SECTION 2

A project contract, a reusable command, scheduled days, colour tiles, a weekly summary and an AI nudge.



END OF SECTION 3

Hardened. A test harness, sub-agents, an agent team that ships a feature under test, and CI.

Real backend, real frontend, real edge cases — streak maths, time zones, idempotency — so the afternoon's harness has something genuine to bite on.



NO ONE GETS STRANDED

Each section starts where the last one finished. If yours doesn't, the reference repo's does.

SMALL COMMITS

Your undo.

A bad turn is one revert away, no drama. git is your safety net inside a section.

CHECKPOINTS

The room's net.

Fall behind? The reference repo holds a working app at the end of every section. Drop into it and keep going.

Reaching for the checkpoint is the design, not a failure.



NO ONE LEFT BEHIND

Everyone starts on a different rung. Everyone leaves higher.

For some, this morning is revision; for everyone, it's a rung up. A core path for all of us, stretch goals for anyone flying. The agentic shift is hardest for the confident ones — they have the most habits to unlearn.



WHAT YOU'LL WALK AWAY WITH

Wherever you start, you leave able to do more.

RUSTY / EARLIER-CAREER

The loop, the core tools, and a working mental model.

You leave able to build.

MID-LEVEL

The discipline: context, reusable workflows, small commits, correctness made mechanical.

You leave able to build reliably.

SENIOR

The architecture of agentic systems: harnesses, sub-agent teams, where human judgement sits.

You leave able to design how your team works with agents.



How the engineer's life has changed

Jade Loyzaga, Canva

SECTION 01 • THE COCKPIT

Vibe Coding

How to fly the agent instead of typing the code.

25 MINUTES • ONE LOOP • TWO TOOLS



ONE BOX, TWO VOICES

ChatGPT's real invention wasn't a smarter model. It was splitting the prompt in two.

WHAT YOU TYPE

Fix the login bug.



*the app
prepends*

WHAT THE MODEL ACTUALLY READS

SYSTEM PROMPT · you never see this

You are a helpful coding assistant.
You have these tools: read, write, bash...

USER PROMPT · the bit you typed

Fix the login bug.

Four words. One box.

This is the only part of the prompt you ever see.

One stream of text. The model cannot tell your words from the builder's — **it just reads the box, top to bottom.**



CONTEXT ENGINEERING

The system prompt isn't one thing. It's a stack, reassembled on every single call.

THE CONTEXT WINDOW

finite

Persona, rules, guardrails

set once by the builder

Memory files

CLAUDE.md / AGENTS.md

Retrieved documents

= RAG

Live tool + server results

= MCP

Everything said so far

conversation history

YOUR PROMPT

the only bit you type

The whole job:

What goes in each layer.

In what order.

And what stays out.

/context

see the stack

Every layer costs space. Fill the window and the model starts guessing.

RAG, MCP, memory, tools — none of them talk to the model. **They just write a better system prompt.**



THE DAY IS A LADDER

Not three workshops. Each rung removes a different crutch.

01

Vibe Coding

Use the tool. Make it work, don't care how.

YOU ARE HERE

02

Vibe Engineering

Make it repeatable. Stop repeating yourself.

03

Agentic Engineering

Stop trusting the agent. Engineer around it.



WHAT VIBE CODING IS

The one-line definition.

Vibe coding is using the tool.

You describe the outcome. The agent writes, runs, and fixes the code. You watch, react, and steer.

THE MINDSET SHIFT

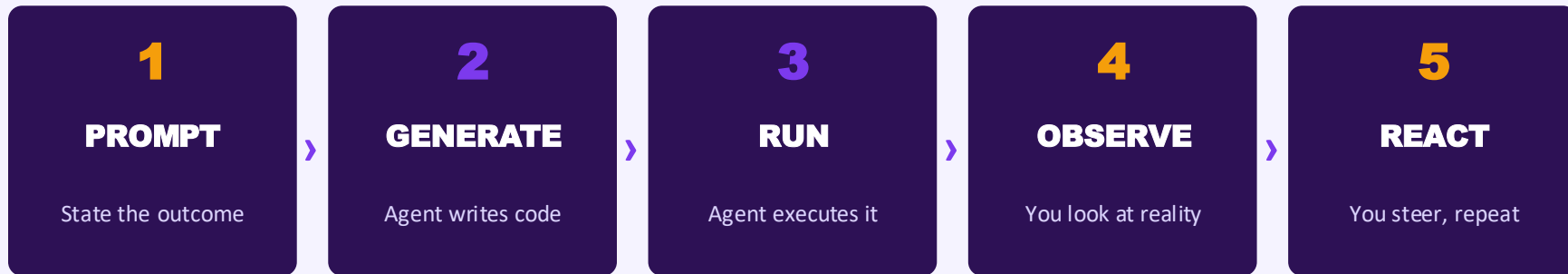
FROM *“How do I write this function?”*

TO *“What do I want to be true when this is done?”*



THE AGENTIC LOOP

The mental model that matters most.



Your job lives in steps 1, 4 and 5. The agent owns 2 and 3. The faster you spin this loop, the more you ship — so keep each turn small.



WHAT THE AGENT ACTUALLY IS

Under the hood.

1

Context window

Everything it can see now: your prompt, files, past turns, tool output. Finite — and it fills up.

2

Tools

How it acts: read/write files, run the shell, run tests, git, search, MCP servers.

3

The loop

Call a tool, read the result, decide the next move, repeat — until the goal is met.

4

Permissions

What it may do without asking. You set the guardrails; it works inside them.

***Not autocomplete.** A program that reasons, acts through tools, and checks its own results in a loop.*



THE ONE RULE FOR THIS MORNING

Resist the keyboard.

The instinct to “just fix it myself” is exactly what we're retraining. When your hands twitch toward the code, that's the moment to type a better prompt instead.

Rusty: easier — you don't recall the syntax anyway.

Confident: this one's for you.



PROMPT BY OUTCOME

Skill #1 — not by step-by-step instructions.



Micromanaging

```
"Open streak.js, add a variable  
count, loop the dates array,  
increment it if..."
```

You're typing code with extra steps. And you have to be right.



Outcome + constraint

```
"Ticking a habit grows its streak.  
Missing a day resets it to zero. Show  
me a plan first."
```

You say WHAT and the rules. The agent picks HOW. No filename, no how.

PLAN REVIEW, NOT PLAN WRITING

Skill #2 — what changed this year.

THEN • 2025

Force a plan. Check the steps — right file, sane order. It often planned clumsily.

NOW • 2026

Decomposition is solid. The risk flips: a clean, confident plan for slightly the wrong problem.

So read for intent and scope, not mechanics:

- Did it understand the real goal, or a plausible lookalike?
- Is it about to touch more than it should?
- What did it assume that you never said?

Turn it on — Claude Code: Shift+Tab cycles to plan, or /plan · Codex: /plan



PERMISSIONS ARE A DIAL

The setting nobody touches. Touch it.



Auto-accept everything

`"Yes to all, it's faster."
Then one shell command, in a
repo that matters.`

*You approved that in advance, and you weren't watching
when it ran.*



Match the dial to the blast radius

`Ask-every-time while the task is new.
Auto-accept only in a sandbox, or a
repo you could afford to delete.`

*Claude Code: Shift+Tab cycles the mode. Codex:
/permissions.*

THE COCKPIT

Skill #3 — four moves on the context window. That is the entire cockpit.

THE MOVE	CLAUDE CODE	CODEx
SEE what it is holding right now	<code>/context</code>	<code>/status</code>
LOAD the exact files the task touches	<code>@path/to/file</code>	<code>@path/to/file</code>
TRIM summarise away what is stale	<code>/compact</code>	<code>/compact</code>
RESET new task, clean window	<code>/clear</code>	<code>/new</code>

***See, load, trim, reset.** Nothing to memorise — but every context problem you hit today is one of these four.*



CONTEXT IS THE FUEL

The gauge tells you which of the four moves to reach for.

Too little

It was never told enough. So it guesses — wrong file, invented convention, confident nonsense.

→ **LOAD** *more of the right thing*

Just right

One task. Only what that task touches. Nothing left over from an hour ago.

→ **HOLD** *live here*

Too much

Slow, and old threads leak into new answers. The window is full, so it starts forgetting.

→ **TRIM** *then RESET*



WHEN IT BREAKS — IT WILL

Skill #4 — report symptoms, don't reach for the file.



Hand-debugging

- Open the file, read the trace, start editing...
- You've grabbed the keyboard again.
- The agent has no idea what you just changed.



Report symptoms

- Paste the exact error text.
- Describe the screen: “streak shows 3, I ticked it, still 3.”
- **Say what you saw, never what to change.**

COMMIT SMALL, SHIP OFTEN

Skill #5 — git is your undo.

Let the agent run git for you:

```
"Commit this with a sensible message."
```

TWO NETS — YOU'LL USE BOTH TODAY

Small commits = your undo. A bad turn is one revert away, no drama.

Checkpoints = the room's safety net. Fall behind? Drop into the reference repo, which holds a working app at the end of every section. Using it is expected, not failure.



WHAT CHANGED IN A YEAR

The ground is moving.

THEN · A YEAR AGO

NOW · 2026

PLANNING

Coax a plan; check its mechanics.

› Agents plan well. You check intent & scope.

CONTEXT

Tiny windows; constant manual pruning.

› Repo-scale memory — but context engineering is now the skill.

AGENTS

One agent, one chat.

› Parallel subagents & agent teams.
(That's this afternoon.)

*The models got better at the mechanics. **The human skill moved up the stack — intent, context, verification.***



FIVE GOLDEN RULES

Carry these into the lab.

1

Resist the keyboard. Direct the loop; don't type the code.

2

Prompt by outcome. Say what and the constraints, not how.

3

Plan review, not writing. Read for intent and scope, not mechanics.

4

Recover by reporting symptoms. Paste the error, describe the screen.

5

Commit small; use checkpoints. Git is your undo. The net is expected.



BY 4PM YOU'LL HARDEN THIS WITH A TEAM OF AGENTS

Let's build.

This morning you learn to fly the loop. You're not behind — the typing got automated; the judgement didn't.



SECTION 02 • MAKE IT REPEATABLE

Vibe Engineering

Stop re-explaining yourself. Make the agent behave the same tomorrow as today — and make good prompts reusable.

20 MINUTES • ONE CONTRACT • TWO TOOLS

THE DAY IS A LADDER

Rung two. You have a working app — now make building it repeatable.

01

Vibe Coding

Use the tool. Make it work, don't care how.

DONE

02

Vibe Engineering

Make it repeatable. Stop repeating yourself.

YOU ARE HERE

03

Agentic Engineering

Stop trusting the agent. Engineer around it.



WHAT VIBE ENGINEERING IS

The one-line definition.

Deliberate, repeatable use of the tool — so it's fast and consistent.

Same agent, same loop as this morning. What changes is discipline: the project **remembers** your conventions, and your best prompts become **shared assets** instead of vanishing into one chat.

THE MINDSET SHIFT

FROM “Let me re-explain how we do things here... again.”

TO “The contract already tells you how we do things here.”



FIVE MOVES THIS SESSION

Each one stops you repeating yourself.

1 **A project contract.** The agent remembers your conventions — written once, read every session.

2 **A reusable workflow.** Your best “add a feature” prompt becomes a command you invoke by name.

3 **Context on purpose.** `/compact`, `/clear`, fresh sessions — keep the agent's desk clean.

4 **Two agents in parallel.** git worktrees let two agents work at once; compare and take a second opinion.

5 **Reach real data.** Connect an external tool over MCP so the agent sees more than your files.



MOVE 1 - THE PROJECT CONTRACT

AGENTS.md / CLAUDE.md — written once, read every session.

WHAT GOES IN IT

- Commands: make dev / test / check
- Architecture: where things live
- Key patterns: weekday encoding,
the fresh-streak rule, idempotency
- The “adding a new field” checklist

WHY IT PAYS OFF

- It follows your conventions without
being told, every session
- New chats start smart, not blank
- One shared asset for the whole cohort
- Codex reads it too, same file

`/init` scaffolds it in both tools. Keep **AGENTS.md** the source of truth; Claude reads it via `@AGENTS.md`.

MOVE 2 - CODIFY A WORKFLOW

A good prompt, saved once, reused forever.

Capture your “add a feature” recipe as a command, the same five steps every time:



Where the command lives

`.claude/commands/` or
`.claude/skills/`

`~/.codex/prompts/`

Invoke it by name

`/add-feature "longest streak"`

Same body in both tools. The prompt stops being something you retype — it's a shared, versioned asset.



MOVE 3 - MANAGE CONTEXT ON PURPOSE

Context is the fuel, spend it deliberately.

One task, one session

Start fresh for a new job so an old thread's assumptions don't leak in.

`/clear` · `new session`

Trim when it bloats

When answers drift or the gauge fills, compact before you continue.

`/compact`

Pin what matters

Point the agent at the exact files; check what it's carrying first.

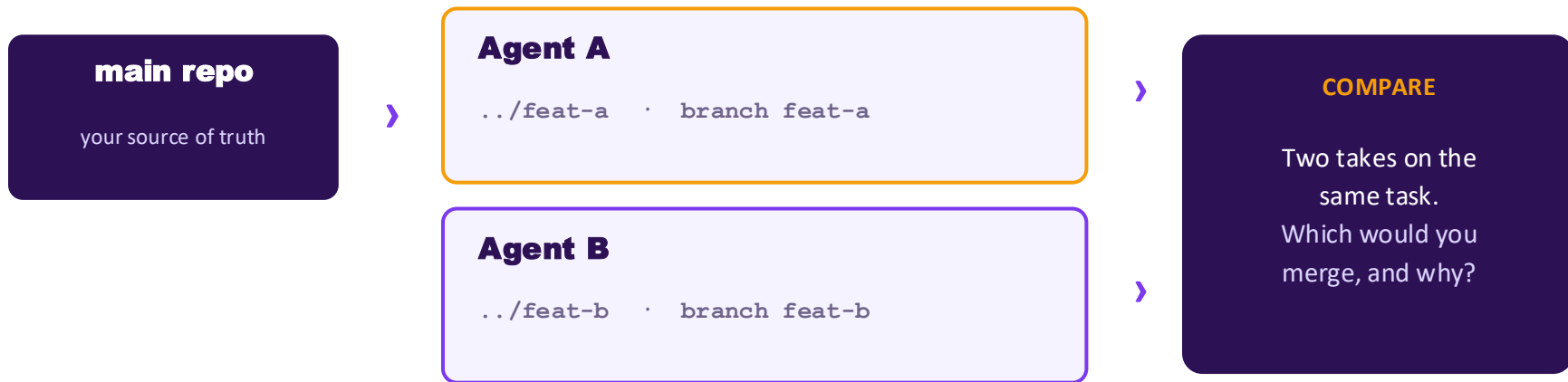
`@path` · `/context (/status)`

A repeatable agent needs a clean desk. Same discipline every task — not a scramble when answers go vague.



MOVE 4 - TWO AGENTS, IN PARALLEL

git worktrees, two agents work the same repo without stepping on each other.



```
git worktree add ../feat-a -b feat-a  - a new branch in its own folder, its own agent.
```

Name = branch = folder. Small, labelled commits in each keep the parallel work mergeable.



MOVE 5 - REACH REAL DATA WITH MCP

Connect the agent to a real tool, not just the files in your repo.

MCP (Model Context Protocol) is a standard connector that lets the agent talk to external systems — your calendar, a database, an API — through one common interface.

AN EXAMPLE FOR TODAY

Connect your calendar and have the agent suggest habit times around your actual schedule, real data, not a guess.

BOTH TOOLS

Claude Code & Codex both read MCP servers.

Codex: [\[mcp_servers\]](#)
You choose what it reaches.

Permissions still apply, connecting a tool is a decision you make, not a door you leave open.



THE FEATURE THAT NEEDS JUDGEMENT

This is where engineering starts, the agent surfaces a decision; you make it.

Add a weekly summary: a short, human paragraph about your week. Where does the text come from?

Static template

- + Instant, offline, free, predictable.
- Same phrasing every single week.

Model call

- + Varied, personal, genuinely warm.
- A dependency, a key, latency, cost.

Plan Mode (Claude: Shift+Tab · Codex: /plan) forces the agent to surface this choice — not decide it silently. **You arbitrate.**

WHERE YOU'LL BE BY LUNCH

Same app, five moves later. Yours, not ours.

- ✓ **A project contract:** AGENTS.md / CLAUDE.md the agent reads every session
- ✓ **A reusable workflow:** your /add-feature command, checked into the repo
- ✓ **Scheduled days and colour tiles:** each habit due only when you choose, as a grid
- ✓ **A weekly summary:** the human paragraph — a decision made on purpose
- ✓ **AI encouragement:** a supportive nudge when a streak is at risk

Fall behind? Drop into the reference repo at `checkpoint-2-features` and keep going. Using the net is expected, not failure.

FIVE RULES FOR THIS SESSION

Carry these into the lab.

- 1 Write it down once.** The contract stops you repeating yourself.
- 2 Save your best prompts.** A good prompt becomes a command, not a memory.
- 3 Keep a clean desk.** Manage context on purpose, every task.
- 4 Get a second opinion.** Two agents beat one, compare before you merge.
- 5 Make the agent surface decisions.** Plan first. You arbitrate; it doesn't decide silently.



BRIDGE TO SECTION 3 · AFTER LUNCH

Would you trust it in front of users?

The app has features now, but nobody has proven it's right. This afternoon we stop hoping: we build the harness, and hand the building to a team of agents.

NEXT · AGENTIC ENGINEERING



SECTION 03 • THE HARNESS

Agentic Engineering

How to trust the work when you can't read every line.

30 MINUTES • FOUR MOVES • ONE TEAM OF AGENTS

THE TOP OF THE LADDER

Two crutches gone. This is the last one: trusting output by eye.

01

Vibe Coding

You stopped writing code by hand.

DONE

02

Vibe Engineering

You stopped repeating yourself.

DONE

03

Agentic Engineering

You stop trusting output by eye.

YOU ARE HERE



WHERE WE GOT TO

Two things are fixed. One is still wide open.

SECTION 1 GAVE YOU

Fast.

The agent writes, runs and fixes. You direct the loop.

SECTION 2 GAVE YOU

Consistent.

A contract, reusable commands, deliberate context.

Neither made it trustworthy.

Would you put this in front of real users on Monday?



WHAT AGENTIC ENGINEERING IS

The one-line definition.

Building systems where **agents do the work**, and the engineering goes into making that **trustworthy**.

THE MINDSET SHIFT

FROM “Did the agent write good code?”

TO “What would prove this code is right — without me reading it?”



READING HARDER DOESN'T SCALE

At agentic speed, line-by-line review is the bottleneck — and it still misses things.

✗ READ EVERY LINE

- The agent out-types you 50 to 1.
- Attention fades; you skim by the third diff.
- A confident, plausible bug reads as correct.

✓ ENGINEER THE PROOF

- Tests the agent must pass every turn.
- Sub-agents whose only job is to check.
- Gates that block a change that fails.

You stop being the line reviewer and become the person who designs how correctness gets proved.



THE FOUR MOVES

Everything in this section is one of these. Nothing else.

1

Build the harness

One command that proves the code still works: lint, types, tests.

2

Split the jobs

Narrow sub-agents: one writes tests, one reviews, one verifies.

3

Run the team

They work in parallel while you orchestrate and arbitrate.

4

Make it non-optional

Hooks and CI, so the discipline survives a bad day.



MOVE 1 - THE HARNESS

One command the agent runs against itself, every single turn.

```
$ make check
  ruff .                # lint - style and dead code
  mypy backend/         # types - wrong shapes
  pytest tests -q       # tests - wrong behaviour
```

WHY ONE COMMAND

The agent can run it, read the failure, and fix itself — without asking you.

IN PLAIN TERMS A harness is just a single command that says yes or no. If it passes, the agent has proof. If it fails, it has a specific error to work from.

The cheapest reliability win there is: the agent self-verifies at the end of every turn.



THE TESTS ARE THE SPEC

Write them before the implementation — and make them fail first.

PROMPT

“Write a pytest suite for the streak logic, covering the edge cases we skipped: a missed scheduled day resets the streak; completing twice in one day is idempotent; a day still in progress doesn't reset it. Run them, show me one failing, then make it pass.”

MISSED DAY

Streak resets to zero — the rule the whole app rests on.

DOUBLE-TICK

Two taps in one day is still one completion. (Idempotent.)

MIDNIGHT

Whose “today” is it? Nothing in this app knows. Record that gap, don't hide it.

Seeing it fail first is the proof the test is real. A test that never failed proves nothing.



MOVE 2 - SUB-AGENTS

Three narrow jobs, each with its own fresh context and minimal tools.

TEST-WRITER

Writes failing tests from the requirement. Never implements.

TOOLS: read + write tests

REVIEWER

Reads the diff, reports bugs as a numbered list with file:line.

TOOLS: read-only – no edits

VERIFIER

Runs make check and the app. Confirms green, or says why not.

TOOLS: read + run shell

```
Claude Code  .claude/agents/reviewer.md      # markdown + frontmatter
Codex        .codex/agents/reviewer.toml  # sandbox_mode = "read-only"
```



WHY NARROW BEATS CAPABLE

The counter-intuitive bit — give each agent less, not more.

✗ ONE AGENT, ALL THE JOBS

It wrote the code, so it reviews its own work kindly. Its context is full of the reasons it thought the code was fine — the same blind spot, twice.

✓ SEPARATE, NARROW AGENTS

A reviewer that never saw the reasoning arrives fresh. And one with no write access can only report problems — it can't quietly "fix" them into hiding.

A reviewer with no write access gives more honest reviews.



HOW THE HANDOFF WORKS

A sub-agent doesn't share your conversation. It gets a brief, and hands back a verdict.

YOU BRIEF IT	IT WORKS ALONE	IT HANDS BACK
Only what you write in the prompt crosses over. It can't see the conversation that led here. CROSSES: <code>the prompt only</code>	Its own context window, its own tool permissions. It fills up, then it's gone. ITS OWN: <code>context + tools</code>	One summary comes back to your session. Not its reasoning, not the forty files it read. RETURNS: <code>one summary</code>

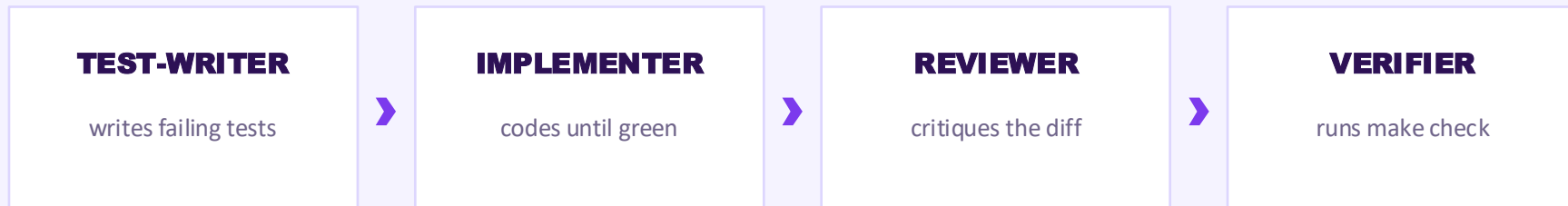
IN PLAIN TERMS Every handoff needs a defined input and output. “Here is the diff, return numbered bugs with file:line and severity” works. “Reviewer, thoughts?” returns mush.

Context isn't shared. It's passed, and only what you pass exists.



MOVE 3 - THE AGENT TEAM

The centrepiece. Four roles in sequence, you at the top.



You orchestrate and arbitrate. You do not type the implementation.

IN PLAIN TERMS You are the tech lead, not the developer. You pick the feature, settle disagreements between agents, and decide when it's done.

If it stalls, the fix is almost never a better prompt — it's a clearer definition of done.



MOVE 4 - MAKE IT NON-OPTIONAL

A hook is a script the tool runs automatically at a defined moment. Not a reminder. Enforcement.

THE CONTRACT

`AGENTS.md`

- Read at the start of a session.
- Followed if the agent is paying attention.
- Quietly forgotten as context fills.

THE HOOK

`.claude/settings.json`

- Runs every single time the event fires.
- Its exit code decides what happens next.
- Can block the action outright.

A contract is a promise. A hook is a fact.



WHEN A HOOK CAN FIRE

Pick the cadence first, then the event. Three rhythms cover everything you'll actually wire.

ONCE PER SESSION	<code>SessionStart</code> <code>SessionEnd</code> Set the scene, or clean up after.
ONCE PER TURN	<code>UserPromptSubmit</code> <code>Stop</code> Gate what goes in, and what counts as finished.
EVERY TOOL CALL	<code>PreToolUse</code> <code>PostToolUse</code> The tight loop. Where guardrails actually live.

Claude Code exposes 30 events, Codex 11. You will use about six of them.



THE SIX WORTH KNOWING

Every one of these exists in both Claude Code and Codex.

EVENT	FIRES WHEN	BLOCKS?	WHAT YOU'D USE IT FOR
<code>SessionStart</code>	a session begins or resumes	No	Brief the agent: branch, ticket, what's failing
<code>UserPromptSubmit</code>	you hit enter, before the model sees it	Yes	Append standards; reject a prompt with a secret in it
<code>PreToolUse</code>	before a tool call runs	Yes	Block edits to protected files; refuse rm in a real repo
<code>PostToolUse</code>	after a tool call succeeds	No	Auto-format the file just written; log the diff
<code>Stop</code>	the agent thinks it has finished	Yes	Run make check. Refuse to stop while it is red
<code>SubagentStop</code>	a sub-agent finishes	Yes	Check the sub-agent actually did its job

“Blocks?” is the column that matters: only some events can stop the thing that is about to happen.



FOUR WORTH WIRING TODAY

Concrete, and all four are useful on TCA Habits before you leave.

THE VERIFIER

Stop

Run make check when the agent says it's done. Red means not done.

THE GUARDRAIL

PreToolUse

Block an edit to the streak logic unless a test changes in the same turn.

THE TIDIER

PostToolUse

Run the formatter on whatever file was just written. Style never reaches review.

THE BRIEFER

SessionStart

Echo the branch and any failing tests, so the agent starts already oriented.

The first two are today's lab. The other two take about a minute each.



THE ONE THING THAT WILL CATCH YOU

A hook signals its decision through the exit code. The numbers are not what you'd guess.

<code>exit 0</code>	Fine, carry on.	Success. Anything on stdout can add context.
<code>exit 2</code>	BLOCK IT.	stderr is handed back to the agent as the reason.
<code>exit 1</code>	Nothing happens.	A non-blocking error. The action proceeds anyway.

Unix says 1 means failure. A policy hook that exits 1 enforces nothing at all.

```
Claude Code  .claude/settings.json + .claude/hooks/      ·  manage with /hooks
Codex        ~/.codex/hooks.json, or [hooks] in config.toml  ·  on by default; [features] hooks =
false turns them off
```



THE FULL CATALOGUE

Reference, not memorisation. Know the six; know this page exists.

CLAUDE CODE · 30 EVENTS

Session	SessionStart · Setup · SessionEnd
Turn	UserPromptSubmit · UserPromptExpansion · Stop · StopFailure
Tools	PreToolUse · PostToolUse · PostToolUseFailure · PostToolBatch · PermissionRequest · PermissionDenied
Agents & tasks	SubagentStart · SubagentStop · TaskCreated · TaskCompleted · TeammateIdle
Context	PreCompact · PostCompact · InstructionsLoaded
Environment	ConfigChange · CwdChanged · FileChanged · WorktreeCreate · WorktreeRemove
MCP & display	Elicitation · ElicitationResult · Notification · MessageDisplay

CODEX · 11 EVENTS

PreToolUse
PostToolUse
PermissionRequest
UserPromptSubmit
Stop
SubagentStart
SubagentStop
SessionStart
SessionEnd
PreCompact
PostCompact

code.claude.com/docs/en/hooks

developers.openai.com/codex/hooks



EVALS - SCORING WHAT TESTS CAN'T

Back to Move 1. The harness proves correctness, but some output has no correct answer.

A TEST asks “is this correct?” and answers pass or fail. **AN EVAL** asks “is this good enough?” and answers with a number.

1

Write a rubric

5 cases, each with what a good answer must contain.

2

Let a model grade

A sub-agent scores each output against the rubric.

3

Gate on the score

CI blocks any change that drops the score.

IN PLAIN TERMS Gate on the score dropping, not on perfection. Block anything that scores worse than what you already had.

If you can't test it, score it. If you can't score it, don't automate it.



MAKING IT STICK AFTER TODAY

Everything so far lives on your laptop. These three make it survive Monday, and survive other people.

CI

GitHub Actions runs the harness and the build on every push and pull request.

SO THAT a teammate who was never in this room still can't break it.

```
.github/workflows/ci.yml
```

HEADLESS

Run the agent one-shot from a script, inside a verification loop.

SO THAT the routine work happens without you in a conversation.

```
claude -p / codex exec
```

SECURITY PASS

Correctness, secrets and licensing, reviewed over the diff before merge.

SO THAT nothing ships that you couldn't explain to someone else.

```
/security-review · /code-  
review / /review
```

Automate the generation, automate the verification, and keep the sign-off human. Accountability doesn't delegate.



THE ONE RULE FOR THIS AFTERNOON

Don't read harder. Build the proof.

Every time you feel the urge to scroll the diff looking for the bug, ask instead: what test, what sub-agent, or what gate would have caught this for me?

Rusty: you never trusted the output by eye anyway — this is your advantage.

Confident: your reading is good. It still isn't fast enough. This one's for you.



IN THE LAB — 60 MINUTES

Four blocks. You orchestrate; you don't type implementation.

A	The harness Tests for the edge cases we skipped. See one fail, then make it pass.	15 min
B	The sub-agents Create the test-writer, the read-only reviewer, and the verifier.	15 min
C	The team Ship a monthly progress ring end to end, under TDD.	20 min
D	Lock it in Hooks, CI, and a security pass. Commit and tag.	10 min

Behind? Drop into the reference repo at checkpoint-2-features and start the block clean.



FIVE GOLDEN RULES

Carry these into the lab.

1 Harness first. Write the check before the feature. Green is the definition of done.

2 Fail it once. A test that has never failed is decoration, not proof.

3 Narrow the agents. One job, minimal tools. The reviewer never gets write access.

4 Orchestrate, don't type. Pick the work, arbitrate disputes, decide when it's finished.

5 Enforce it in the tool. Hooks and CI, not good intentions.



WHEN YOU GET BACK TO YOUR OWN REPO

Today was a toy app on a projector. This is the order to do it in on work that matters.

1

Build the harness

TOMORROW

One repo you own. One command: lint, types, tests. Nothing clever, and see it fail once.

2

Add one agent, one hook

THIS WEEK

A read-only reviewer sub-agent, plus one hook that blocks the mistake your team keeps making.

3

Make it the team's

THIS MONTH

Harness into CI, an eval for anything fuzzy, and walk one teammate through the same two steps.

Start where the diffs already make you nervous.



BY 4PM: A HARDENED APP AND A TEAM THAT BUILT IT

Let's harden it.

You ran the loop this morning. You made it repeatable before lunch. Now you build the system that produces the work and proves it — the same skills scale from a weekend project to a production service. Only the size of the harness changes.